Week 4, End-to-end machine learning pipeline, Part 2

CAPP 30254, Machine Learning for Public Policy

The end-to-end machine learning pipeline

Let's say I want to design a model that is able to predict how individuals are most likely to commute – by car, public transit, walking, biking, or remote work – based on demographic and geographic features.

classification – multiclass
demographic, geographic data

What steps do I need to take in order to achieve this? I can follow the *end-to-end machine learning pipeline* – the process of building a machine learning model.

The pipeline:

Define my problem → collect data → clean the data → feature selection and engineering → split the data

Ex. one-hot encoding – why?
N E S W directions
1 2 3 4
E ≠ N
W ≠ 2·E
ordered → not true/ nonsense
distance
Better: E: [0, 1, 0, 0]

Ex. "too close" features – How to tell?
– domain knowledge
"too close": same info
– correlation
$corr(x_i, x_j) = \frac{cov(x_i, x_j)}{\sigma_{x_i} \sigma_{x_j}}$
> 0.9 or < −0.9 drop
only works for numeric features
What happens?
Linear models: unstable coefficients

Split the data

Split the data into training, validation, and testing sets.

Typical percentages:

- Training: 60-80%
- Validation: 10-20%
- Testing: 10-20%

~ 80% when you have a large dataset

~ 60/20/20 when you want more data for evaluation

With limited data:

skip validation set: train: 80 - 90 %

test: 10 - 20 %

Use cross validation to tune hyperparameters

on the train set

Split the data

Split the data into training, validation, and testing sets.

Data is usually split using random sampling.

data randomly assigned to train, valid., test

If your data is imbalanced, use *stratisfied sampling*. When you split your data using stratified sampling, each of the train, validation, and test sets will have the same proportion of each class as the original dataset.

90% class A, 10% class B

Random split might create:

train: mostly A

test: mostly B → unfair evaluation

Stratisfied sampling:

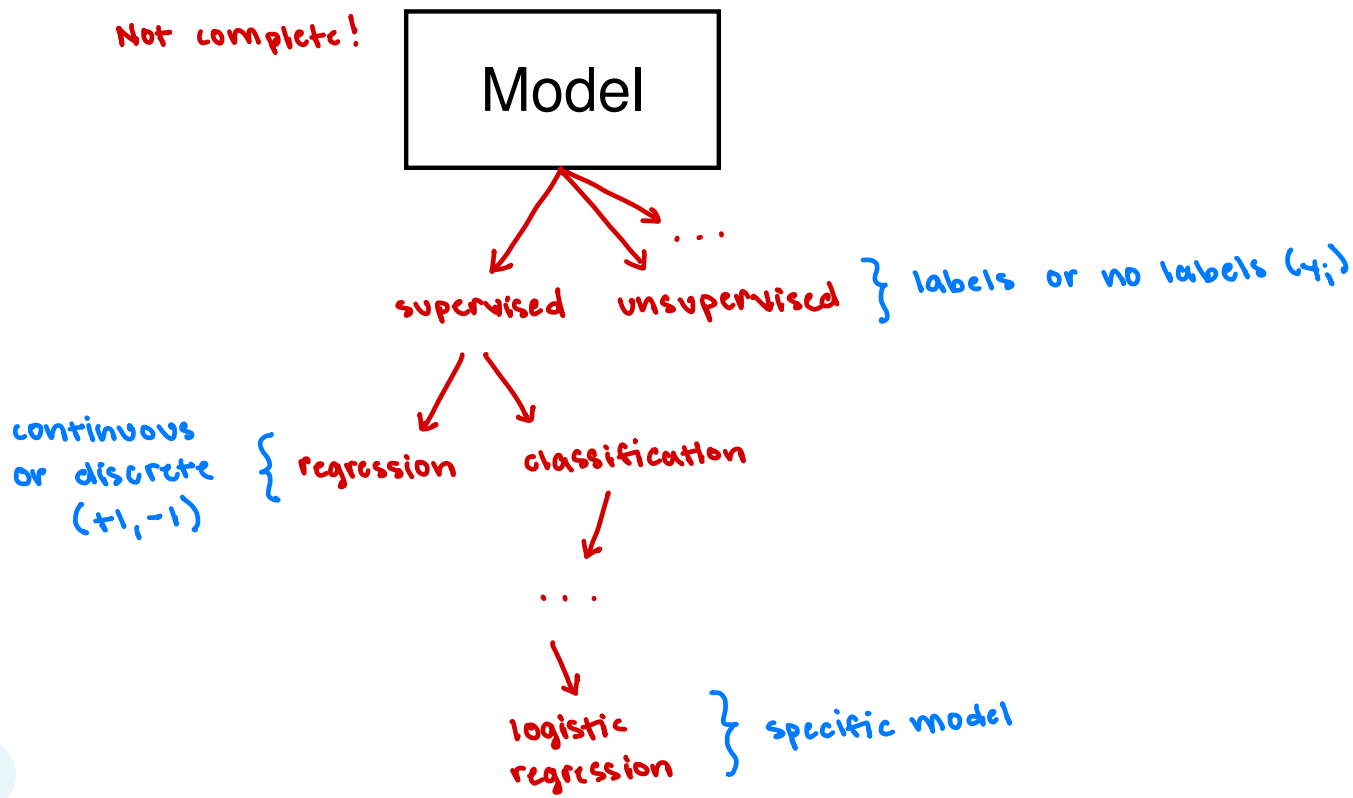
guarantees each of train, valid, test has the same distribution of each class

class A, class B

Model selection

Select an appropriate model for your problem.

Is your problem a classification or regression problem? Is your problem supervised or unsupervised?



Add a regulizer?

Recall ridge regression, which adds an L_2 penalty term to the least squares loss function:

L_2 penalty

$$\ell(\mathbf{w}) = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{p} w_i^2$$

bias term not included

just want to shrink the weights

Adding a penalty term to a loss function is known as *regularization*. can add to any loss function

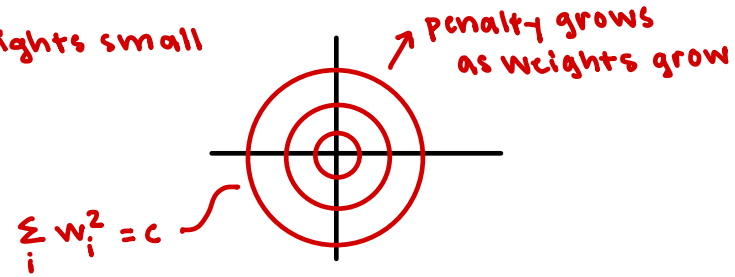
Adding an L_2 term to a model penalizes large weights, which prevents overfitting.

large weights ↔ overfitting

because the model is trying too hard to fit the data

- more complex
- sensitive to noise

L_2 keeps weights small



Add a regulizer?

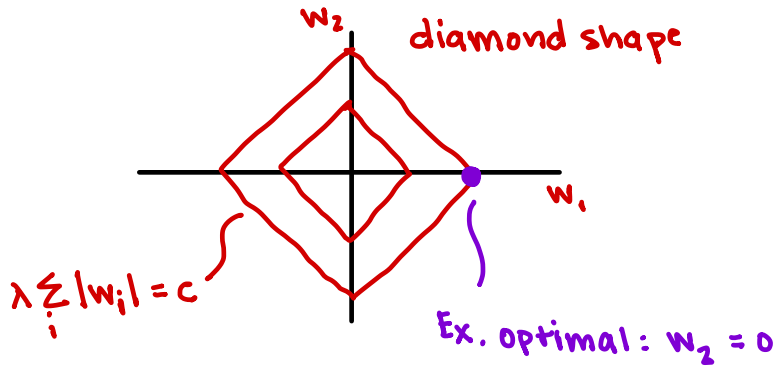
Which regulizer? L_1, L_2, other?

Adding an L_1 term to a model also penalizes large weights. With this penalty term, some weights become exactly zero.

L_2: ridge → small weights

L_1: LASSO → small, zero weights (sparse models)

L_1 penalty: $\lambda \sum_{i=1}^{p} |w_i|$



Train the model and tune hyperparameters

The next step in the pipeline is to train the model and tune hyperparameters.

Tune:

better estimate of the performance of the model configuration → cross-validation: steps 2 and 3
- "training set" → current fold
- "validation set" → rest of the folds
- repeat

The process is:

1. ■ Choose a hyperparameter value
2. ■ Train a model using those values on the training set
3. ■ Evaluate it on the validation set
   - ■ Repeat for many hyperparameter values
   - ■ Pick the best model

What if there are multiple hyperparameters?
A. - random search
B. - grid search (exhaustive)

start with A.
refine with B.

Train:

Finally, retrain your model using the entire training set.

Never use your test set to train your model!

on validation or w/ cross-validation →

Grid search:
- choose possible values for each hyperparam
- train with that combination
- pick the best

Evaluate the model

After you've trained your model, we can evaluate its performance on the test set. Remember, the test set was never used during training or tuning, so it will give you a more honest estimate of how well your model will perform.

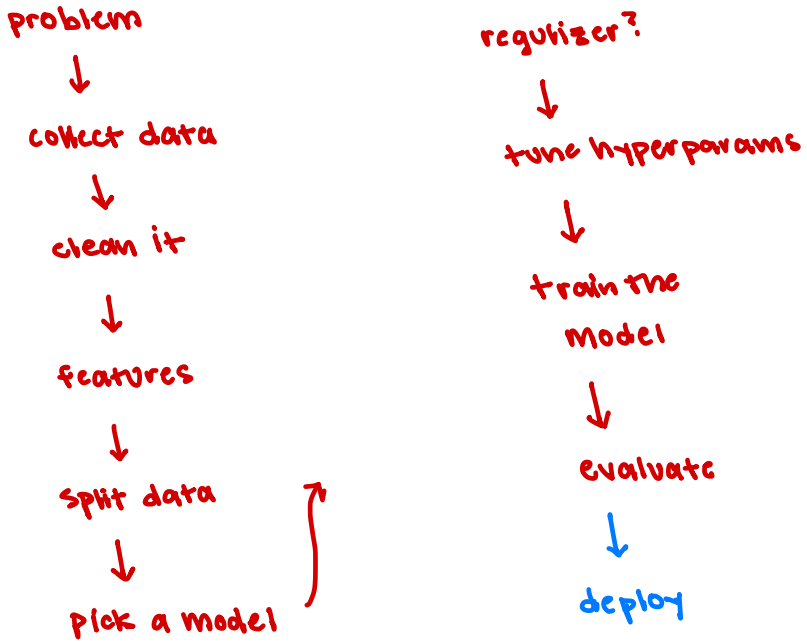
Use evaluation metrics like accuracy, precision, F1, etc.

Evaluate using the right metric for your problem

Metric	What it measures	Example
Accuracy $= \frac{1}{n}\sum_i 1(y_i = \hat{y}_i)$	Proportion of correct predictions	Predicting voter turnout
Precision $\frac{TP}{TP+FP}$ (P)	Of predicted positives ($\hat{y}_i = +1$), how many were correct?	Predicting welfare fraud minimize accusing innocent people
Recall $\frac{TP}{TP+FN}$ (R)	Of actual positives ($y_i = +1$), how many were found	Predicting eligibility for emergency housing want all who qualify to be identified minimize FN
F1 Score $\frac{2 \cdot P \cdot R}{P+R}$	Harmonic mean of precision and recall	Predicting students at risk of dropping out balance catching true cases and avoiding false alarms

End-to-end machine learning pipeline

The entire pipeline is:



[1]

[1] Generated by GPT-4, DALL-E 3 after the prompt: “Hi, could you please make a minimalistic logo for a machine learning class...”